Unit I : Introduction To Systems Programming And Assemblers

Language Processor and Compiler Construction

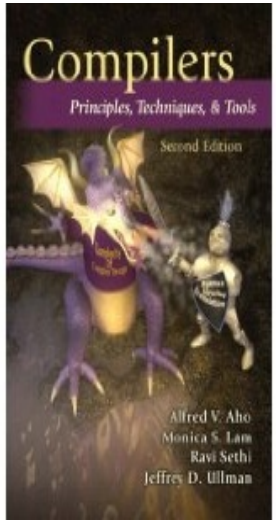
Manisha Mali
manisha.mali@viit.ac.in
Department of Computer Engineering

BRACT'S, Vishwakarma Institute of Information Technology, Pune-48

(An Autonomous Institute affiliated to Savitribai Phule Pune University)
(NBA and NAAC accredited, ISO 9001:2015 certified)

Books

- D. M. Dhamdhere, Systems Programming and Operating Systems, Tata McGraw-Hill, ISBN 13:978-0-07-463579-7, Second Revised Edition
- Alfred V. Aho, Ravi Sethi, Jeffrey D. Ullman, Compilers Principles, Techniques and Tools, Addison Wesley, ISBN:981–235–885 - 4, Low Price Edition
- J. J. Donovan, Systems Programming, McGraw-Hill, ISBN 13:978-0-07-460482-3, Indian Edition
- J R Levin, T Mason, D Brown, “Lex and Yacc", O'Reilly, 2000 ISBN 81-7366-061-X



Course Prerequisite

- Fundamentals of Data structures
- Theory of Computation : DFA, NFA, Regular expressions, Grammars
- Concepts of Operating Systems
- Programming Languages

Course Outcomes

1. Develop hypothetical assembler. (Apply)
2. Illustrate macro processors, linkers and loaders. (Understand)
3. Implement lexical analyser using LEX tool (Apply)
4. Build parser using YACC tool (Apply)
5. Construct the intermediate code representations (Apply)
6. Demonstrate code optimization and code generation concept (Understand)

Content

Part – I : Introduction

- **Need of System Software,**
- **Components of System Software**
- **Language Processing Activities**
- **Fundamentals of Language Processing**
- **Interpreter**

Part- II : Assemblers

- Elements of Assembly Language Programming
- A simple Assembly Scheme
- Pass structure of Assemblers
- Design of Two Pass Assembler

Introduction

- Software
  - Application software usually used by end-user
    - It is concerned with the solution of some problem, using the computer as a tool, instead of how computers actually work.
  - System software
    - System software consists of a variety of programs that support the operation of a computer (ex: text editor, compiler, debugger)
    - One characteristic in which most system software differ from application software is <u>machine dependency</u>
    - A system software programmer must know the target machine structure

Types of System Software

- Operating System
- Language Processors
- Device Drivers

Basic Features and Design Options

- Fundamental features
  - Basic functions and characteristics should remain essentially the same, regardless of what machine is being used.
- Major design options
  - There is no single “right” way of doing things; a software designer needs to be aware of the available options in order to make intelligent decisions

System Software and Machine Architecture

- Machine dependent system software
  - System programs are to support the operation and use of the target computer.
  - The difference between different machine
    - Machine code
    - Instruction formats
    - Addressing mode
    - Registers
- Machine independent system software
  - General design and logic is basically the same:
    - Code optimization
    - General design and logic of an assembler

System Software

- The system software includes
  - **Assembler**
  - **Linker**
  - **Loader**
  - **Macro processor**
  - **Text editor**
  - **Compiler**
  - **Operating system**
  - **Debugging system**
  - **Source Code Control System**
  - **(optional) Database Management System**

Programming Languages

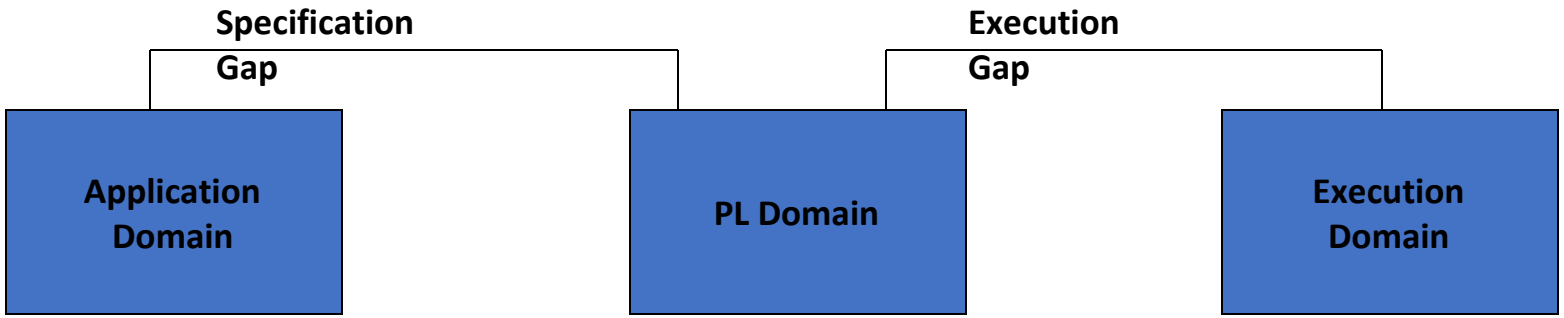
- **Machine language:**
- **It is computer's native language having a sequence of zeroes and ones (binary). Different computers understand different sequences. Thus, hard for humans to understand:**
- **e.g.0101001...**

- **Assembly language:**
- **It uses mnemonics for machine language. In this each instruction is minimal but still hard for humans to understand:**
- **e.g. ADD AH, BL**

- **High-level languages:**
- **FORTRAN, Pascal, BASIC, C, C++, Java, etc.**
  Each instruction composed of many low-level instructions, closer to English. It is easier to read and understand:
- **e.g. hypot = sqrt(opp*opp + adj * adj);**

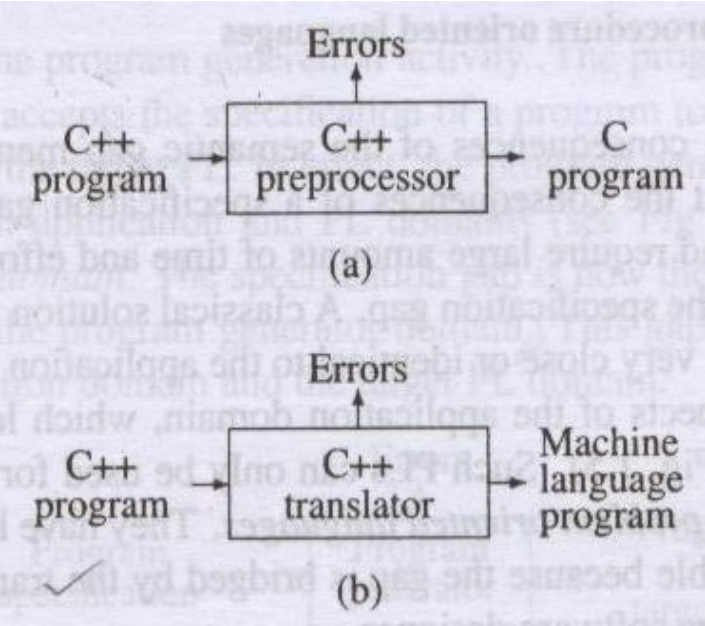
Language processors

- A Language processor is a software which bridges a specification or execution gap



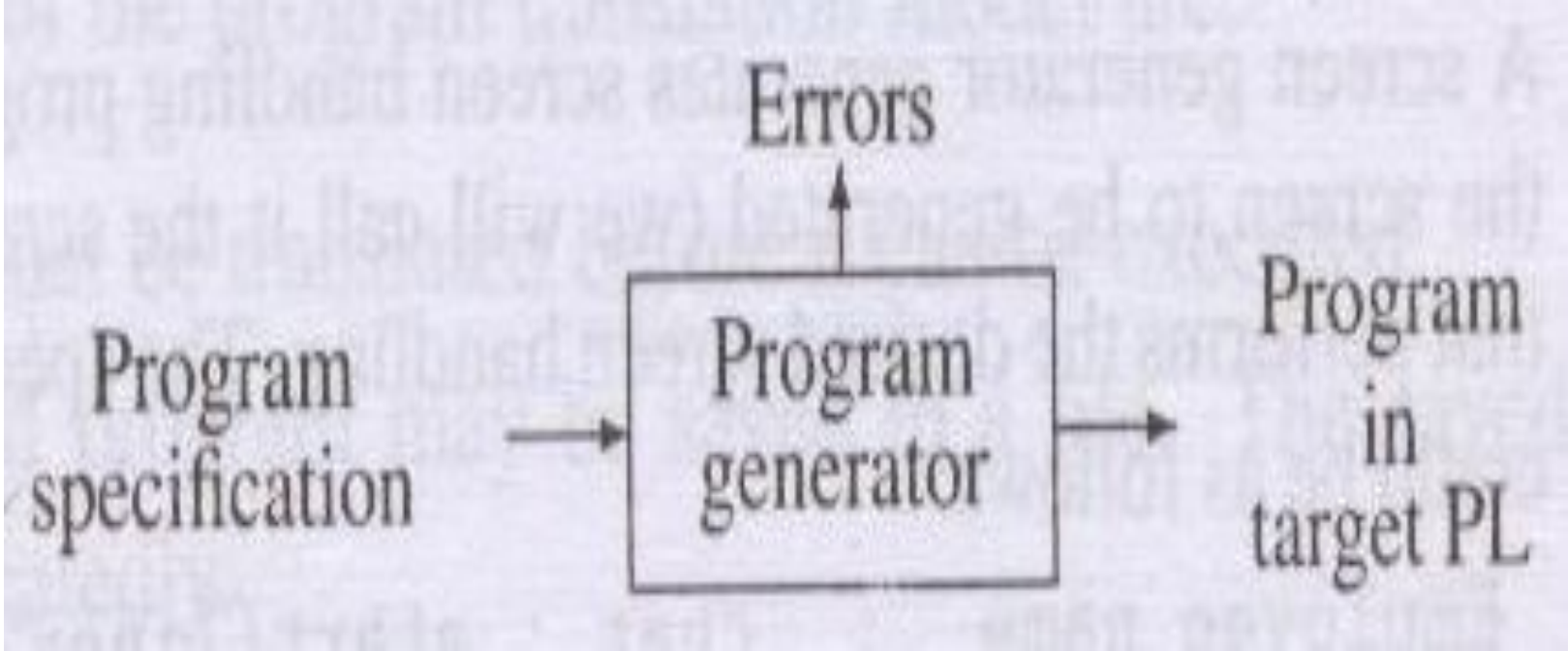
Language processors

- A language translator
- A detranslator
- A preprocessor
- A language migrator
- Interpreter



- **1. Language Translator**
- A **language translator** is system software that converts a program written in one programming language (source language) into another language (target language), usually machine language or intermediate code, so that it can be executed by a computer system.
  Examples: Assembler, Compiler, Interpreter.
- **2. Detranslator**
- A **detranslator** is system software that converts a machine-level or low-level program back into a higher-level or symbolic form to aid in program analysis, debugging, or understanding.
  Example: Disassembler converting machine code into assembly language.
- **3. Preprocessor**
- A **preprocessor** is a language processing tool that performs preliminary transformations on the source program before it is handed over to the compiler or assembler.
  It handles tasks such as macro expansion, file inclusion, conditional compilation, and symbol substitution.
  Example: C preprocessor directives like #define, #include.
- **4. Language Migrator**
- A **language migrator** is a software tool that translates programs written in one programming language into an equivalent program in another programming language, typically to support platform upgrades, modernization, or technology migration.
  Example: Converting legacy COBOL programs into Java or C#.

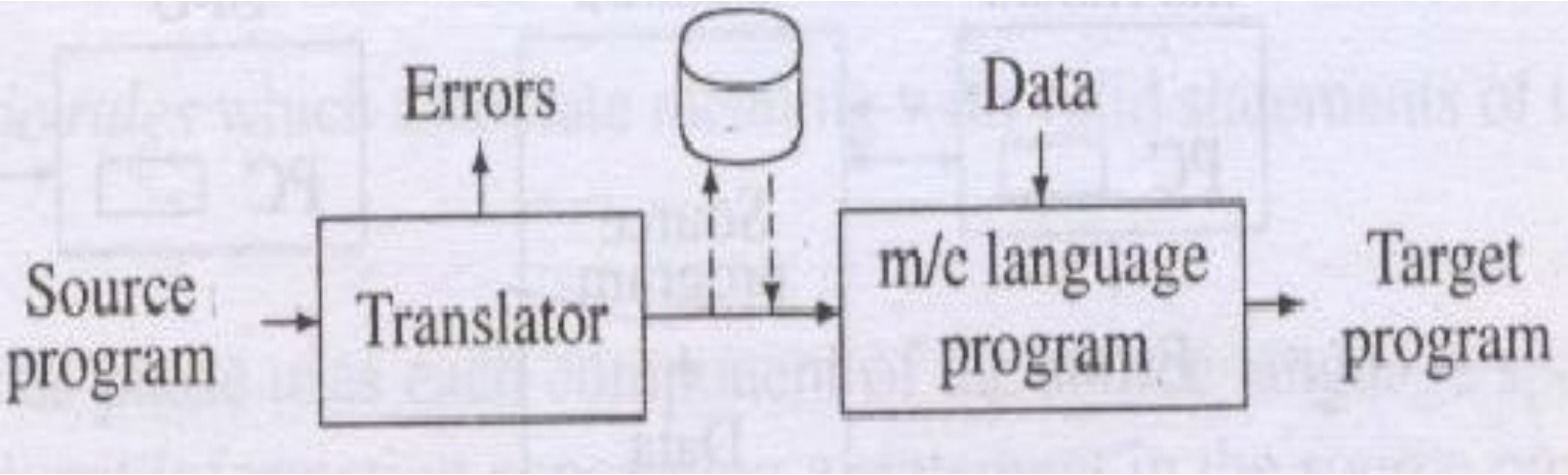
Program generation activities



Program execution activities

i Program translation

ii Program interpretation



Program Translation vs Program Interpretation

Program Translation

- Converts the **entire source program** into **machine code** before execution
- Generates a **separate object/executable file**
- Errors are reported **after complete translation**
- Execution is **faster** once translation is complete

Examples: Compiler, Assembler

Program Interpretation

- Translates and executes the program **statement by statement**
- **No object code** is generated
- Errors are detected **immediately during execution**
- Execution is **slower** due to repeated translation

Examples: Python Interpreter, JavaScript Interpreter

Fundamentals of language processing

- Language processing
  = Analysis of source program
  + synthesis of target program

- Lexical rules
- Syntax rules
- Semantic rules

Lexical, Syntax, and Semantic Rules

Lexical Rules

- Define how **characters are grouped into tokens**
- Specify valid **identifiers, keywords, constants, operators**
- Implemented by the **Lexical Analyzer (Scanner)**
- Ignore **white spaces and comments**
  Example: int sum = a + b; → tokens: int, sum, =, a, +, b

Syntax Rules

- Define the **grammatical structure** of statements
- Specify how tokens are combined using **formal grammar**
- Checked by the **Syntax Analyzer (Parser)**
- Detect **structural errors**
  Example: Missing semicolon, incorrect if statement format

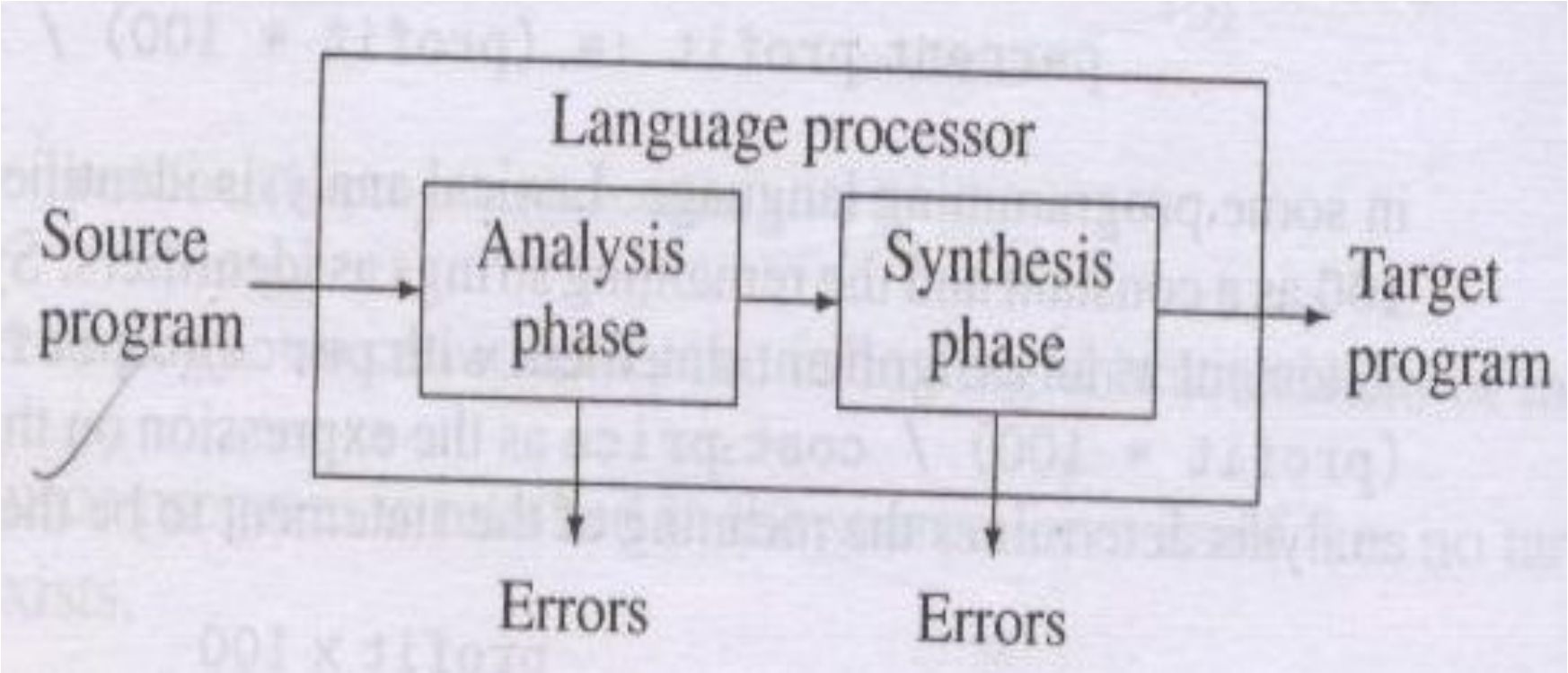
Semantic Rules

- Define the **meaning and logical correctness** of statements
- Ensure **type compatibility, scope, and valid operations**
- Checked by the **Semantic Analyzer**
  Example: int x; x = "Hello"; → type mismatch error

Fundamentals of language processing (Contd.)

a. Phases of a language processor
b. Forward reference
c. Passes of a language processor
d. Intermediate representation

A. Phases of a language processor



B Forward reference

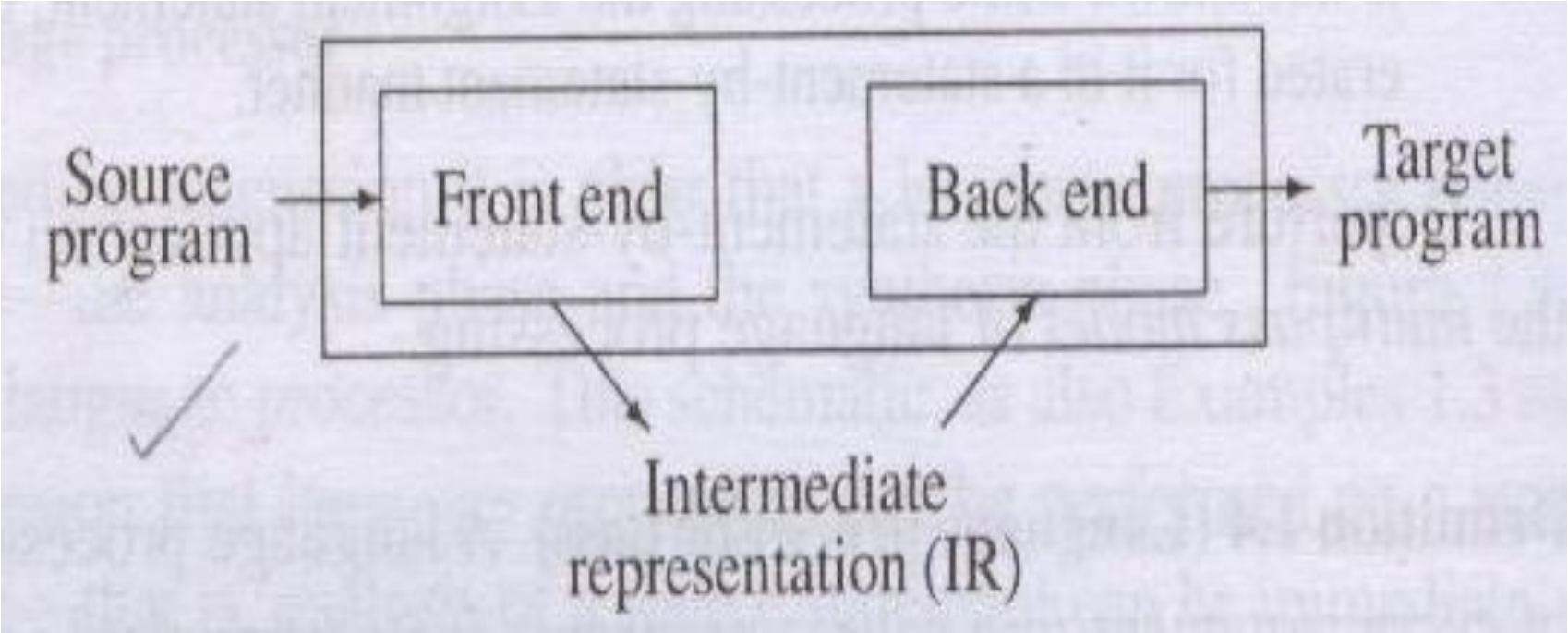
- A FR of a program entity is a reference to the entity which precedes its definition in the program

C. Passes of a language processor

- Pass I
  - Performs analysis of the source program and note relevant information
  - Scans source program once
  - Generates machine code immediately
  - **Limitation:**
  - Cannot handle forward references easily

- Pass II
  - Perform synthesis of target program

D. Intermediate representation



Fundamentals of language specification

- Programming language grammar
  - Terminal symbols, alphabets and strings
  - Classification of grammar

Type	Name	Recognized By
Type-0	Unrestricted Grammar	Turing Machine
Type-1	Context-Sensitive Grammar	Linear Bounded Automaton
Type-2	Context-Free Grammar	Pushdown Automaton
Type-3	Regular Grammar	Finite Automaton

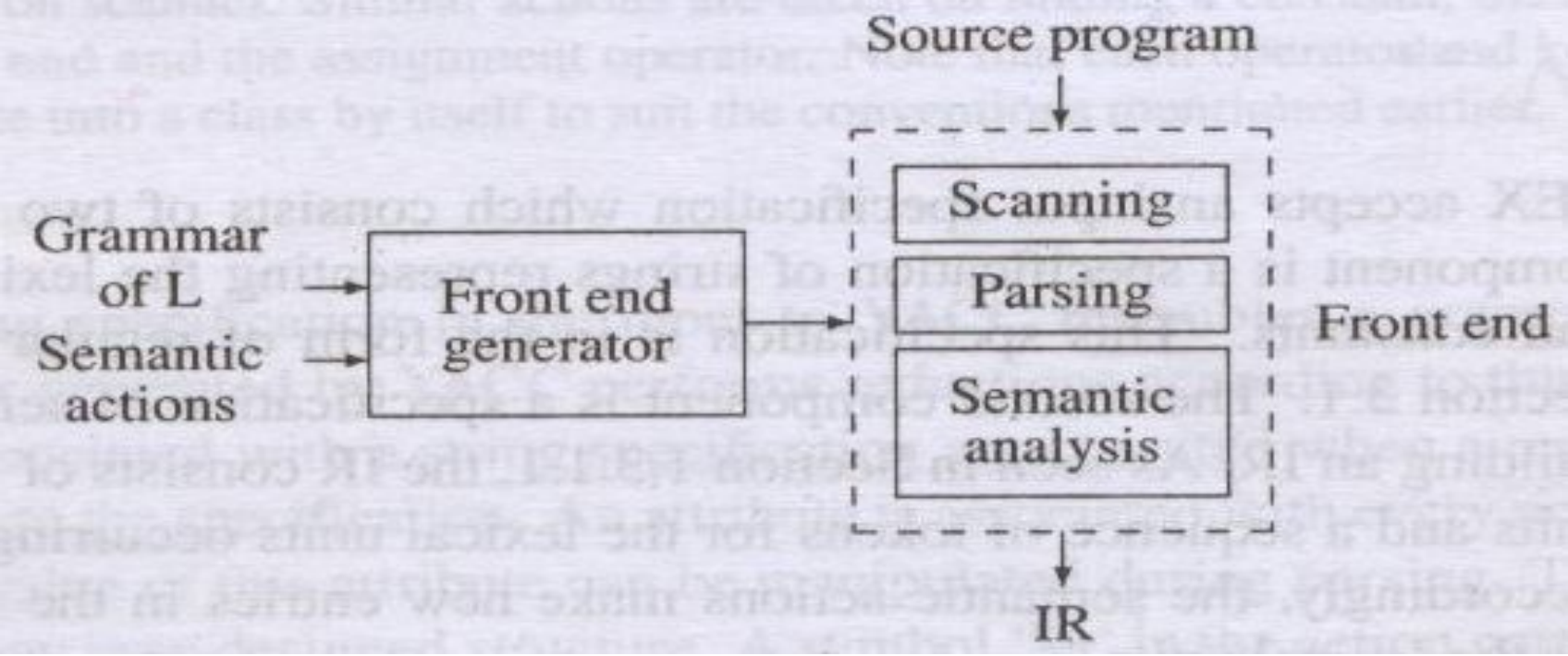
- **1. Programming Language Grammar**
- A **programming language grammar** is a **formal specification of the syntax** of a programming language.
  It defines **how valid programs are formed** using rules.
- Formally, a grammar **G** is defined as:
- $G = (V, T, P, S)$
- where:
- **V** → Set of non-terminal symbols
- **T** → Set of terminal symbols
- **P** → Set of production rules
- **S** → Start symbol
- **Purpose of grammar**
- Define program structure
- Help in syntax analysis (parsing)
- Used by compilers and interpreters

- **2. Terminal Symbols, Alphabets, and Strings**
- **(a) Alphabet (Σ)**
- A **finite, non-empty set of symbols**.
- Example:
- $\Sigma = \{a, b\}$

(b) Terminal Symbols

- Symbols that **cannot be replaced further**
- Appear in the **final output (string/program)**
- Example:
  Keywords, identifiers, operators (if, +, id)

Language processor development tools

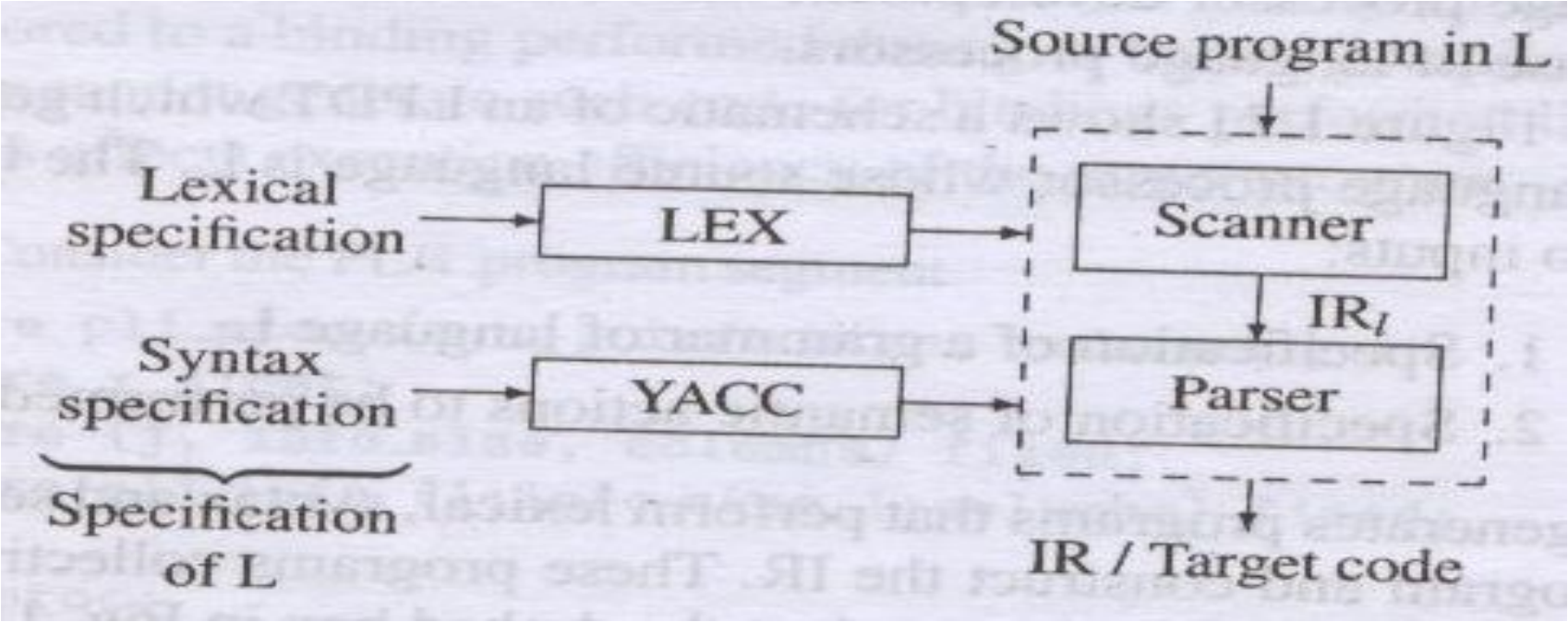


Language processor development tools

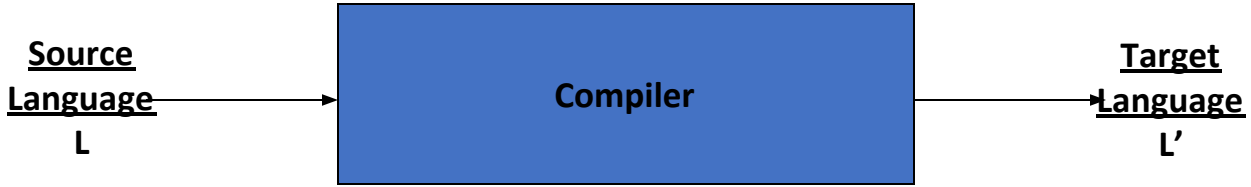
Aspect	Scanning (Lexical Analysis)	Parsing (Syntax Analysis)
Definition	Scanning is the process of **reading source code characters** and grouping them into **tokens**	Parsing is the process of **analyzing token sequences** to check **syntactic structure**
Compiler Phase	First phase of compiler	Second phase of compiler
Input	Source code (character stream)	Tokens from scanner
Output	Tokens (id, keyword, operator, literal)	Parse tree / syntax tree
Main Objective	Identify **valid tokens**	Verify **grammatical correctness**
Grammar Used	Regular grammar / regular expressions	Context-Free Grammar (CFG)
Error Type Detected	Lexical errors (invalid character, malformed token)	Syntax errors (missing semicolon, wrong structure)
Complexity	Simple and fast	More complex
Data Structure Used	Finite Automata	Stack / Parse Table
Example Tools	Lex, Flex	Yacc, Bison

- Semantic analysis performs **context-sensitive checks**, including:
- **Type checking**
  - Ensures operands and operators are compatible
- **Declaration checking**
  - Variables must be declared before use
- **Scope resolution**
  - Checks correct variable binding (local/global)
- **Function call validation**
  - Number and type of arguments must match
- **Type conversion (coercion)**
  - Applies implicit or explicit type conversions

Language processor development tools (Contd)



What is a Compiler?



Java
C++
FORTRAN

JVM
Intel
Pentium
MIPS
IBM's CELL

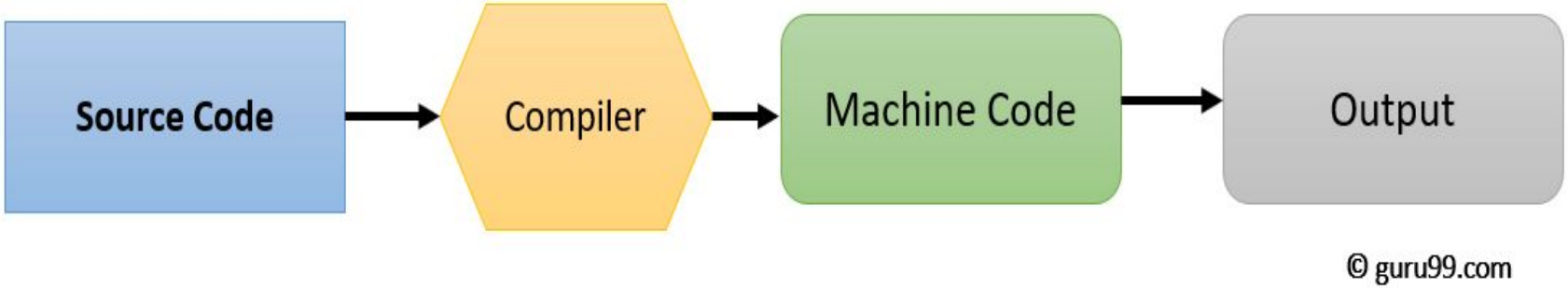
A compiler translates text from a source language, L,
to
A target language, L'.

Interpreter

An interpreter is a computer program, which coverts each high-level program statement into the machine code. This includes source code, pre-compiled code, and scripts. Both compiler and interpreters do the same job which is converting higher level programming language to machine code. However, a compiler will convert the code into machine code (create an exe) before program run. Interpreters convert code into machine code when the program is run.

Ref : https://www.guru99.com/difference-compiler-vs-interpreter.html

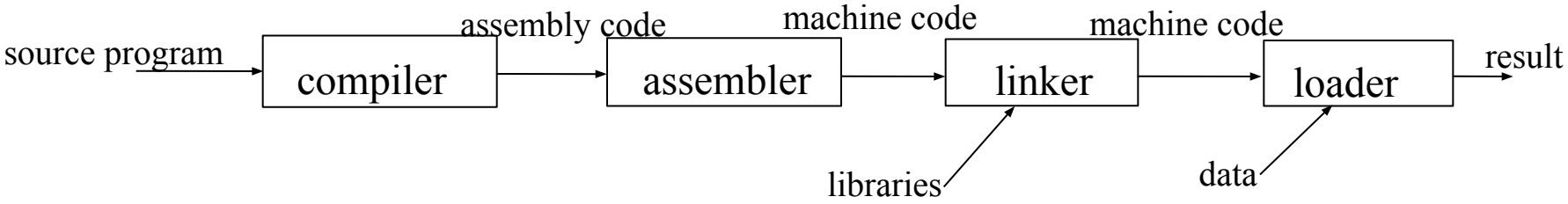
How Compiler Works



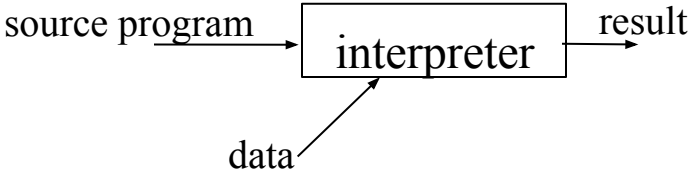
How Interpreter Works

Architecture

Compiler:



Interpreter:



Java uses both a compiler (javac) and an interpreter (java)

Role of Interpreter

- The interpreter converts the source code line-by-line during RUN Time.
- Interpret completely translates a program written in a high-level language into machine level language.
- Interpreter allows evaluation and modification of the program while it is executing.
- Relatively less time spent for analyzing and processing the program
- Program execution is relatively slow compared to compiler

Key Difference between Compiler and Interpreter

- Compiler transforms code written in a high-level programming language into the machine code, at once, before program runs, whereas an Interpreter coverts each high-level program statement, one by one, into the machine code, during program run.
- Compiled code runs faster while interpreted code runs slower.
- Compiler displays all errors after compilation, on the other hand, the Interpreter displays errors of each line one by one.
- Compiler is based on translation linking-loading model, whereas Interpreter is based on Interpretation Method.
- Compiler takes an entire program whereas the Interpreter takes a single line of code.

Many Other Translators

Source Language	Translator	Target Language
LaTeX	Text Formater	PostScript
SQL	database query optimizer	Query Evaluation Plan
Java	javac compiler	Java byte code
Java	cross-compiler	C++ code
English text	Natural Lang Understanding	semantics (meaning)
Regular Expressions	JLex scanner generator	a scanner in Java
BNF of a language	CUP parser generator	a parser in Java

Challenges

- Many variations:
  - many programming languages (eg, FORTRAN, C++, Java)
  - many programming paradigms (eg, object-oriented, functional, logic)
  - many computer architectures (eg, MIPS, SPARC, Intel, alpha)
  - many operating systems (eg, Linux, Solaris, Windows)

References

- D. M. Dhamdhere, Systems Programming and Operating Systems, Tata McGraw-Hill, ISBN 13:978-0-07-463579-7, Second Revised Edition
- Alfred V. Aho, Ravi Sethi, Jeffrey D. Ullman, Compilers Principles, Techniques and Tools, Addison Wesley, ISBN:981–235–885 - 4, Low Price Edition
- J. J. Donovan, Systems Programming, McGraw-Hill, ISBN 13:978-0-07-460482-3, Indian Edition
- J R Levin, T Mason, D Brown, “Lex and Yacc", O'Reilly, 2000 ISBN 81-7366-061-X
- https://www.guru99.com/difference-compiler-vs-interpreter.html

Thank you!